

1과목 - 응용 SW 기초 기술 활용 요약

운영체제 운용 기법

1. 일괄처리: 일정량 또는 일정기간 동안 데이터를 모아서 한꺼번에 처리
2. 실시간처리: 데이터 발생 즉시 처리하여 결과를 산출하는 방식
3. 다중 프로그램: 하나의 CPU와 주기억장치를 이용하여 여러 개의 프로그램을 동시에 처리
4. 다중 처리: 여러개의 CPU와 주기억장치를 이용
5. 시분할: 여러명의 사용자가 사용하는 시스템에서 번갈아가면서 처리(라운드로빈 방식이라고도 함)

비선점 스케줄링

- 이미 할당된 CPU를 다른 프로세스가 강제로 빼앗아 사용할 수 없는 기법
- 종류: FCFS, SJF, 우선순위, HRN, 기한부 등

비선점 스케줄링 종류

1. FCFS: 먼저 도착한 순서에 따라 처리
2. SJF: 실행 시간이 가장 짧은 프로세스를 먼저 처리
3. HRN: 대기시간과 서비스 시간을 이용하는 기법(SJF의 단점을 보완) - $(\text{대기시간} + \text{서비스 시간}) / \text{서비스 시간}$

선점 스케줄링 종류

1. Round Robin: 시분할 시스템을 위해 고안된 방식으로 FCFS 알고리즘을 선점 형태로 변형한 기법

교착상태의 필요 충분 조건

1. 상호배제(Mutual Exclusion): 한번에 한 개의 프로세스만이 공유 자원을 사용할 수 있어야 함
2. 점유와 대기(Hold and Wait): 최소한 하나의 자원을 점유하고 있으면서 다른 프로세스에 할당되어 사용하고 있는 자원을 추가로 점유하기 위해 대기하는 프로세스가 있어야 함
3. 비선점(No Preemption): 이미 점유된 자원을 강제로 빼앗을 수 없어야 함
4. 순환 대기(Circular Wait): 프로세스들이 서로가 점유하고 있는 자원을 기다리는 순환적인 형태로 대기하고 있어야 함

워킹 셋 / 구역성

- 워킹 셋(Working Set): 프로세스가 일정 시간 동안 자주 참조하는 페이지들의 집합을 의미
- 구역성(Locality): 프로세스가 실행되는 동안 주기억장치를 참조할 때 일부 페이지만 집중적으로 참조하는 성질이 있다는 이론

페이징 기법

- 가상 기억장치에 보관되어 있는 프로그램과 주기억장치의 영역을 동일한 크기로 나눠 주기억장치의 영역에 적재시켜 실행하는 기법
- 페이지의 위치 정보를 가지고 있는 페이지 맵 테이블이 필요함

세그먼테이션 기법

- 가상 기억장치에 보관되어 있는 프로그램을 다양한 논리적인 단위로 나눈 후 주기억장치에 적재시켜 실행하는 기법
- 세그먼트가 존재하는 위치정보를 가진 맵 테이블이 필요

페이지 교체 알고리즘 - FIFO

- 각 페이지가 주기억장치에 적재될 때마다 그때의 시간을 기억시켜 가장 먼저 들어와서 가장 오래 있었던 페이지를 교체하는 기법
- 벨레이디 모순이 발생

순차파일

- 레코드가 키 순서대로 편성되어 취급이 용이
- 파일에 레코드를 삽입, 삭제, 수정하는 경우 파일 재구성을 위해 전체를 복사해야 하므로 시간이 많이 소요 됨

직접 파일

- 레코드에 특정 기준으로 키가 할당되며, 해싱 함수를 이용하여 이 키에 대한 보조기억장치의 물리적 상대 주소를 계산한 후 해당 주소에 레코드를 저장

색인 순차 파일

- 기본 영역에 있는 레코드들의 위치를 찾아가는 색인이 기록되는 영역

디렉터리 구조

- 1단계: 하나의 디렉터리에서 관리
- 2단계: 중앙에 마스터 파일 디렉터리가 있고 그 아래 사용자별 디렉터리가 존재
- 트리: 하나의 루트 디렉터리와 여러 개의 종속 디렉터리로 구성
- 비순환 그래프: 하위 파일이나 하위 디렉터를 공동으로 사용할 수 있는 것으로 사이클이 허용되지 않는 구조

UNIX의 특징

- 시분할 시스템을 위해 설계된 대화식 운영체제
- 다중 사용자, 다중 작업 지원
- 트리구조

Kernel

- UNIX의 가장 핵심적인 부분으로 주기억장치에 적재된 후 상주하면서 실행됨
- 하드웨어를 보호하고 프로그램들과 하드웨어 간의 인터페이스 역할을 담당

Shell

- 사용자의 명령어를 인식하여 프로그램을 호출하고 수행하는 명령어 해석기
- 시스템과 사용자간의 인터페이스를 담당

모뎀

- 컴퓨터나 단말장치로부터 전송되는 디지털 데이터를 아날로그 회선에 적합한 아날로그 신호로 변환

DSU

- 컴퓨터나 단말장치로부터 전송되는 디지털 데이터를 디지털 회선에 적합한 디지털 신호로 변환

코덱

- 아날로그 데이터를 디지털 통신 회선에 적합한 디지털 신호로 변환하는 변환기

주파수 분할 다중화기(FDM)

- 통신 회선의 주파수를 여러 개로 분할하여 여러 대의 단말장치가 동시에 사용할 수 있도록 한 것

시분할 다중화기(TDM)

- 통신 회선의 대역폭을 일정한 시간 폭으로 나누어 여러 대의 단말장치가 동시에 사용할 수 있도록 한 것

HDLC

- 비트 위주의 프로토콜로 각 프레임에 데이터 흐름을 제어하고 오류를 보정할 수 있는 비트열을 삽입하여 전송
- 프레임 구조
 - 시작 플래그
 - 주소부: 송,수신국을 식별하기 위해 사용
 - 제어부: 프레임의 종류를 식별하기 위해 사용
 - 정보부: 실제 정보 메시지가 들어있는 부분
 - FCD: 프레임 내용에 대한 오류 검출을 위해 사용되는 부분
 - 종료 플래그

자동 반복 요청(ARQ)

- 정지-대기(Stop-and-Wait)
- Go-Back-N: 오류가 발생한 블록 이후의 모든 블록을 재전송
- 선택적 재전송: 오류가 발생한 블록만 재전송
- 적응적: 데이터 블록 길이를 채널의 상태에 따라 그때그때 동적으로 변경하는 방식

TCP/IP 계층 구조

- 응용 계층: 응용 프로그램 간의 데이터 송,수신 제공
- 전송 계층: 호스트들 간의 통신 제공
- 인터넷 계층: 데이터 전송을 위한 주소 지정, 경로 배정 제공
- 네트워크 액세스 계층: 실제 데이터를 송,수신하는 역할

OSI 참조 모델

데이터 링크 계층

- 두 개의 인접한 개발 시스템들 간에 신뢰성 있고 효율적인 정보 전송을 할 수 있도록 시스템 간 연결 설정과 유지 및 종료를 담당
- HDLC, LAPB, LLS, MAC, LAPD, PPP, BSC 등의 표준이 있음

네트워크 계층

- 개발 시스템 간의 네트워크 연결을 관리하는 기능과 데이터의 교환 및 중계 기능
- 경로 설정, 데이터 교환 및 중계, 트래픽 제어, 패킷 정보 전송을 수행

전송 계층

- 논리적 안정과 균일한 데이터 전송 서비스를 제공함으로써 종단 시스템 간에 투명한 데이터 전송, 연결 해제 기능을 함
- 주소 설정, 다중화, 오류 제어, 흐름 제어를 수행
- TCP, UDP 등의 표준이 있음

표현 계층

- 응용 계층으로부터 받은 데이터를 세션 계층으로 보내기 전에 통신에 적당한 형태로 변환하고 세션 계층에서 받은 데이터를 응용 계층에 맞게 변환하는 기능을 함

응용 계층

- 사용자가 OSI 환경에 접근할 수 있도록 서비스를 제공
- TELNET, FTP, SMTP, SNMP, HTTP 등의 표준이 있음

응표세전내데물

패킷 교환 방식

- 메시지를 일정한 길이의 패킷으로 잘라서 전송하는 방식
- 대량의 데이터 전송 시 전송 지연이 많아짐
- 전송 시 교환기, 회선 등에 장애가 발생하여도 다른 정상적인 경로를 선택하여 우회할 수 있음

가상 회선 방식

- 가상 통신 회선을 미리 설정하여 송,수신지 사이의 연결을 확립한 후에 설정된 경로를 따라 패킷들을 순서적으로 운반하는 방식

네트워크 관련 장비

- 허브: 가까운 거리의 컴퓨터들을 연결하는 장치, 각 회선을 통합적으로 관리하며 신호 증폭 기능
- 리피터: 물리 계층의 장비로 전송되는 신호를 재생
- 브리지: 데이터 링크 계층의 장비로 LAN가 LAN을 연결하거나 LAN 안에서의 컴퓨터 그룹을 연결
- 라우터: 네트워크 계층의 장비로 동종의 LAN과 LAN 연결 및 경로 선택, 다른 LAN이나 LAN과 WAN을 연결
- 게이트웨이: 프로토콜 구조가 전혀 다른 네트워크의 연결을 수행하는 장비로 세션 계층, 표현 계층, 응용 계층 간을 연결

1과목 - 애플리케이션 설계

애자일 개발 4가지 핵심 가치

- 프로세스와 도구보다는 개인과 상호작용에 더 가치를 둠
- 방대한 문서보다는 실행되는 SW에 가치를 둠
- 계약 협상보다는 고객과 협업에 더 가치를 둠
- 계획에 따르기보다는 변화에 반응하는 것에 더 가치를 둠

객체지향 방법론

- 기계의 부품을 조립하듯이 객체들을 조합하는 방법론
- 요구분석 > 설계 > 구현 > 테스트 및 검증 > 인도

HIPO

- 시스템 분석 및 설계나 문서화할 때 사용되는 기법으로 시스템 실행 과정인 입력, 처리, 출력의 기능을 나타냄
- 하향식 소프트웨어 개발을 위한 도구
- HIPO Chart의 종류
 - 가시적
 - 총체적
 - 세부적

UML

- 시스템 개발 과정에서 시스템 개발자와 고객 또는 개발자 상호간의 의사소통이 원활하게 이루어지도록 표준화한 대표적인 객체지향 모델링 언어

실체화(Realization) 관계

- 사물이 할 수 있거나 해야 하는 기능으로 서로를 그룹화 할 수 있는 관계를 표현함
- 한 사물이 다른 사물에게 오퍼레이션을 수행하도록 지정하는 의미적 관계

일반화(Generalization) 관계

- 하나의 사물이 다른 사물에 비해 더 일반적으로 구체적인지 표현

구조적(정적) 다이어그램의 종류

- 클래스 다이어그램
- 객체 다이어그램
- 컴포넌트 다이어그램
- 배치 다이어그램
- 복합체 구조 다이어그램
- 패키지 다이어그램

행위(동적) 다이어그램의 종류

- 유스케이스 다이어그램
- 순차 다이어그램
- 커뮤니케이션 다이어그램
- 상태 다이어그램
- 활동 다이어그램
- 상호작용 개요 다이어그램
- 타이밍 다이어그램

유스케이스 다이어그램의 구성 요소

- 시스템/시스템 범위
- 액터: 시스템과 상호작용을 하는 모든 외부 요소
 - 주액터: 시스템을 사용함으로써 이득을 얻는 대상으로 주로 사람이 해당
 - 부액터: 주액터 목적 달성을 위해 시스템에 서비스를 제공하는 외부 시스템
- 유스케이스: 사용자가 보는 관점에서 시스템이 액터에게 제공하는 서비스 또는 기능을 표현한 것
- 관계: 액터와 유스케이스 사이의 관계로 연관, 포함, 확장, 일반화 등이 있음

클래스 다이어그램

- 클래스가 수행할 수 있는 동작으로 함수라고도 함

순차 다이어그램의 개요

- 시스템이나 객체들이 메시지를 주고받으며 시간의 흐름에 따라 상호작용을 그림으로 표현한 것
- 순차 다이어그램에서 수직 방향은 시간의 흐름을 나타냄

소프트웨어 아키텍처 뷰

- 유스케이스 뷰: 시스템 외부 사용자의 관점
- 논리적 뷰: 설계자의 관점
- 구현 뷰: 개발자의 관점
- 프로세스 뷰: 시스템 통합자의 관점
- 배포 뷰: 테스터의 관점

아키텍처 패턴의 장점

- 시행착오를 줄여 개발 시간을 단축시키고, 고품질의 소프트웨어를 생산할 수 있음
- 검증된 구조로 개발하기 때문에 안정적인 개발이 가능

- 의사소통이 간편해짐
- 유지보수가 쉬움

파이프-필터 패턴

- 데이터 스트림 절차의 각 단계를 필터 컴포넌트로 캡슐화하여 파이프를 통해 데이터를 전송하는 패턴
- 필터 간 데이터 이동 시 데이터 변환으로 인한 오버헤드가 발생

MVC 패턴

- 모델: 서브시스템의 핵심 기능과 데이터를 보관
- 뷰: 사용자계에 정보를 전달
- 컨트롤러: 사용자로부터 입력된 변경 요청을 처리하기 위해 모델에게 명령을 보냄

추상 클래스

- 구체 클래스에서 구현하려는 기능들의 공통점만 모아 추상화한 것
- 인스턴스 생성 불가능, 구체 클래스가 추상 클래스를 상속받아 구체화한 후 구체 클래스의 인스턴스를 생성

캡슐화

- 데이터와 데이터를 처리하는 함수를 하나로 묶는 것을 의미
- 캡슐화된 객체는 외부 모듈의 변경으로 인한 파급 효과가 적음, 인터페이스가 단순화됨
- 객체 간의 결합도가 낮아짐

상속

- 이미 정의된 상위 클래스의 모든 속성과 연산을 한위 클래스가 물려받는 것

다형성

- 객체가 연산을 수행하게 될 때 하나의 메시지에 대해 각각의 객체가 가지고 있는 고유한 방법으로 응답할 수 있는 능력
- 오버로딩: 메소드의 이름은 같지만 인수를 받는 자료형과 개수를 달리
- 오버라이딩: 상위 클래스에서 정의한 메소드와 이름은 같지만 메소드 안의 실행코드를 달리함

디자인 패턴

- 각 모듈의 세분화된 역할이나 모듈들 간의 인터페이스와 같은 코드를 작성하는 수준의 세부적인 구현방안을 설계할 때 참조할 수 있는 전형적인 해결 방식 또는 예제를 의미
- 디자인 패턴 유형: 생성, 구조, 행위

아키텍처와 디자인 패턴의 차이점

- 아키텍처 패턴은 디자인 패턴보다 상위 수준의 설계에 사용
- 아키텍처 패턴: 전체 시스템 구조를 설계하기 위함
- 디자인 패턴: 서브시스템에 속하는 컴포넌들과 그 관계를 설계하기 위함

생성 패턴

- 추상팩토리
- 빌더
- 팩토리 메소드
- 프로토타입
- 싱글톤

구조 패턴

- 어댑터
- 브리지
- 컴포지트
- 데코레이트
- 퍼싸드
- 플라이웨이트
- 프록시

행위 패턴

- 책임 연쇄
- 커맨드
- 인터프리터
- 반복자
- 중재자
- 메멘토
- 옵저버
- 상태
- 전략
- 템플릿 메소드
- 방문자

1과목 - 테스트 및 배포

소프트웨어 테스트 순서

1. 단위테스트
2. 통합테스트
3. 시스템 테스트
4. 인수 테스트

하향식 통합 테스트

- 상위 모듈부터 시작해서 하위 모듈로 점진적으로 내려가며 통합 테스트를 수행하는 방식
- 시스템의 전체 구조를 초기에 검증 가능
- 실제 사용자 시나리오와 유사한 테스트 가능
- 하위 모듈이 없을 경우 스텝을 사용
 - 스텝이란 아직 구현되지 않은 하위 모듈을 대신하는 가짜 모듈

상향식 통합 테스트

- 하위 모듈부터 먼저 테스트하고 점차 상위 모듈과 결합하는 방식
- 기능 단위의 안정성을 먼저 확보
- 상위 모듈이 없을 경우 드라이버를 사용
 - 드라이버란 아직 구현되지 않은 상위 모듈을 대신하는 테스트용 호출 프로그램

결합 관리 프로세스

1. 결합 관리 계획
2. 결합 기록
3. 결합 검토
4. 결합 수정
5. 결합 재확인
6. 결합 상태 추적 및 모니터링 활동
7. 최종 분석 및 보고서 작성

사용자 인터페이스의 구분

- CLI: 명령어 입력
- GUI: 그래픽 요소
- NUI: 음성, 제스처, 터치 등

- VUI: 음성 명령
- OUI: 모든 사물과 사용자 간의 상호작용

빌드 자동화 도구

- 빌드란 소스 코드 파일들을 컴파일한 후 여러 개의 모듈을 묶어 실행 파일로 만드는 과정
- Ant, Make, Maven, Gradle, Jenkins

1과목 - 정보시스템 기반 기술 용어

OWASP(Open Web Application Security Project)

- 웹 정보 노출이나 악성코드, 스크립트, 보안이 취약한 부분을 연구하는 비영리 단체임